

Table of Contents

Chapter 3: Object-Oriented Programming (OOP) Using Python	2
Introduction: The Core Shift — From Linear Scripts to Modular Real-World Objects	2
3.1 Object-Oriented Programming (OOP) Using Python.....	3
The Hook (Story Mode)	3
The Explanation.....	3
The Practical Walkthrough.....	4
Interactive Stop-Point	4
Quick Recap	4
3.2 Core Concepts of OOP.....	5
3.2.1 Classes and Objects — Representing Real-World Entities in Code.....	5
The Hook (Story Mode)	5
The Explanation.....	5
Memory Blueprint: How Objects Live in RAM.....	6
Interactive Stop-Point	7
Quick Recap	7
3.2.2 Encapsulation	7
The Hook (Story Mode)	7
The Explanation.....	8
The Practical Walkthrough.....	10
Interactive Stop-Point	10
Quick Recap	10
3.2.3 Inheritance — Reusing and Extending Functionality.....	10
The Hook (Story Mode)	10
The Explanation.....	11
The Practical Walkthrough.....	12
Interactive Stop-Point	14
Quick Recap	14
3.2.4 Polymorphism.....	14
The Hook (Story Mode)	14
The Explanation.....	14
The Practical Walkthrough.....	17
Interactive Stop-Point	18
Quick Recap	18
Chapter Glossary (Quick Reference)	19
Chapter Summary	19

Chapter 3: Object-Oriented Programming (OOP) Using Python

Introduction: The Core Shift — From Linear Scripts to Modular Real-World Objects

Alright, future software architects — let's talk about a moment that changes how you think about code forever.

Up until now, your Python programs have probably looked like a recipe. Step one, do this. Step two, do that. Variables float around loosely. Functions get called on data that lives somewhere else, far away, in another part of the script. This is called **procedural programming** — programming as a straight line of instructions.

But here's the problem. Real life isn't a straight line. Real life is made of *things* — cars, students, bank accounts, video game characters, Instagram profiles. Each of these "things" carries its own data (a car has a color, a speed, a fuel level) and its own behavior (a car can accelerate, brake, honk). When you write procedural code to model these "things," your data and your logic drift apart. Programs get tangled. Bugs multiply. Teams of programmers start stepping on each other's toes.

Object-Oriented Programming (OOP) fixes this by letting you bundle data and behavior together into one unit, modeled after real-world things. Instead of writing loose functions that operate on loose variables, you build **blueprints** (classes) for real-world entities, and then you create **actual instances** (objects) from those blueprints.

This is the single biggest mental shift in your programming journey so far. It is completely normal if it feels strange at first — almost every programmer on Earth had this exact same "wait, what is `self`?" moment. By the end of this chapter, you won't just understand OOP. You'll think in objects.

Let's build.

3.1 Object-Oriented Programming (OOP) Using Python

The Hook (Story Mode)

Rewind to 1967, Norway. Two computer scientists, Ole-Johan Dahl and Kristen Nygaard, were trying to simulate something very physical: ships colliding at sea. Procedural code made this a nightmare — every ship's speed, position, and cargo had to be tracked separately, with functions reaching across the whole program to grab the right numbers for the right ship.

So Dahl and Nygaard invented something new: a language called **Simula 67**. Instead of tracking "ship 1 speed" and "ship 2 speed" as separate loose variables, they created a **"ship" blueprint** — a bundle of data and behavior that could be copied to make as many ships as they wanted, each keeping track of its own information.

They had just invented the **class** and the **object**. A decade later, Alan Kay took this idea and used it to build **Smalltalk**, one of the first languages with a graphical, window-based interface — the direct ancestor of the desktop and phone screens you use today. That accidental-looking idea, born from simulating colliding ships, is the same idea you're about to learn.

The Explanation

Procedural Programming vs. Object-Oriented Programming

Procedural Programming	Object-Oriented Programming
Code is a sequence of steps	Code is a collection of interacting objects
Data and functions are separate	Data and functions are bundled together (in a class)
Hard to reuse for a new "thing"	Easy to reuse — just create a new object
Small bugs can spread everywhere	Bugs stay contained inside one class
Good for short scripts	Good for large, evolving, real-world software

Think of procedural code like a single long shopping list taped to your fridge. Object-oriented code is like having a labeled drawer for every ingredient — flour in the flour drawer, sugar in the sugar drawer — each with instructions for how to use it.

Benefits of OOP: Modularity, Reusability, and Easier Maintenance

- **Modularity** — Your program is broken into self-contained pieces (classes). Each piece can be built, tested, and fixed independently, like Lego bricks.
- **Reusability** — Once you build a `Student` class, you can create a hundred different student objects from it, or reuse that class in a completely different program.

- **Easier Maintenance** — If there's a bug in how a bank account calculates interest, you fix it in *one place* — the `BankAccount` class — and every object made from it is instantly fixed.

One-sentence summary: OOP organizes your program the way the real world is organized — into self-contained "things" that carry their own data and know how to act on it.

The Practical Walkthrough

Imagine you're building a simple game with 100 players. In procedural code, you might need 100 separate variables for health, and a giant function full of `if` statements to figure out whose health to update. In OOP:

```
class Player:
    def __init__(self, name, health):
        self.name = name
        self.health = health

    def take_damage(self, amount):
        self.health -= amount
        print(f"{self.name} now has {self.health} HP.")

# Create 3 independent players from the SAME blueprint
p1 = Player("Zara", 100)
p2 = Player("Malik", 100)
p3 = Player("Aisha", 100)

p1.take_damage(20)    # Only Zara's health changes
```

What just happened conceptually? One blueprint (`Player`), three completely independent objects, each managing its *own* health. No giant tangled `if` chain required.

Interactive Stop-Point

Pause & Think: You're building a library system. In procedural style, you'd have separate lists like `book_titles = []`, `book_authors = []`, `book_available = []`, all lined up by index number. What could go wrong if someone accidentally sorts `book_titles` but forgets to sort `book_authors` the same way? How would bundling this data into a `Book` class prevent that entire category of bug?

Quick Recap

Procedural code separates data and logic into a long line of instructions; OOP bundles data and logic together into reusable, self-contained blueprints called classes — making large programs modular, reusable, and far easier to maintain.

3.2 Core Concepts of OOP

OOP rests on four pillars: **Classes & Objects**, **Encapsulation**, **Inheritance**, and **Polymorphism**. Let's build each one from the ground up.

3.2.1 Classes and Objects — Representing Real-World Entities in Code

The Hook (Story Mode)

Think about your smartphone. Somewhere in a factory in Shenzhen, an engineer designed a **blueprint**: screen size, chip type, camera specs, battery capacity. That blueprint itself is not a phone you can hold — it's just a design. But when the factory manufactures an actual unit, gives it a serial number, and charges its battery to 87%, *that* is a real, physical, individual phone in your hand.

The blueprint is the **class**. Your actual phone, with its own unique IMEI number and its own current battery percentage, is the **object**.

The Explanation

- **Class** — a blueprint that defines what attributes (data) and methods (behavior) every object of this type will have. Defined with the `class` keyword.
- **Object** — a specific instance created from a class, with its own actual values.
- **Attribute** — a variable that belongs to an object (e.g., `name`, `battery_level`).
- **Method** — a function that belongs to a class and defines what objects of that class can *do*.
- **Instantiation** — the act of creating an object from a class.
- **Constructor** (`__init__`) — a special method that runs automatically the moment an object is created. It sets up the object's starting attributes.
- `self` — a reference to "this specific object." It's how a method knows *which* object's data to work with.

ASCII UML Class Diagram:

```
+-----+
|      Student      |
+-----+
| + name            |
| + age             |
| + grade           |
+-----+
| + __init__(name, age) |
| + display()        |
+-----+
```

Defining a Class in Python:

```
class Student:
    def __init__(self, name, age):
        self.name = name      # attribute
        self.age = age        # attribute

    def display(self):
        print(f"Student: {self.name}, Age: {self.age}")
```

About `self` — don't be scared of it. Think of `self` as a name tag every object wears. When you write `self.name = name` inside a method, you are saying: "Whichever object called this method, put this value on *its own* name tag, not on anyone else's." Python passes `self` automatically — you never type it in when you call the method, only when you define it.

Instantiating Objects and Accessing Attributes/Methods:

```
student1 = Student("Amina", 17)
student2 = Student("Bilal", 18)

student1.display()    # Student: Amina, Age: 17
student2.display()    # Student: Bilal, Age: 18

print(student1.name)  # Directly access an attribute: Amina
```

Memory Blueprint: How Objects Live in RAM

When Python runs `student1 = Student("Amina", 17)`, here is what happens, step by step:

Step	Code Executed	What Happens in RAM
1	<code>Student("Amina", 17)</code> is called	Python allocates a brand-new empty object in memory, at some address — let's imagine <code>0x001A</code>
2	<code>__init__(self, name, age)</code> runs	<code>self</code> is automatically bound to the new object at <code>0x001A</code> ; <code>name="Amina"</code> , <code>age=17</code> are passed in
3	<code>self.name = name</code> executes	The object at <code>0x001A</code> now stores an attribute <code>name</code> with value <code>"Amina"</code>
4	<code>self.age = age</code> executes	The object at <code>0x001A</code> now stores an attribute <code>age</code> with value <code>17</code>
5	<code>student1 = ...</code>	The variable <code>student1</code> in your script becomes a <i>label</i> pointing at address <code>0x001A</code>

RAM Memory Box Map:

```
student1 ----> [ 0x001A: Student object ]
                  name: "Amina"
                  age: 17

student2 ----> [ 0x00FB: Student object ]
                  name: "Bilal"
                  age: 18
```

Notice: `student1` and `student2` point to **two completely separate boxes in memory**.

Changing `student1.age` will never touch `student2.age`. This is the whole point of objects — isolated, independent data.

Interactive Stop-Point

Pause & Think — 3.2.1: If you create two objects `student1 = Student("Amina", 17)` and `student2 = Student("Amina", 17)` — same name, same age — does `student1 == student2` evaluate to `True` or `False` in Python?

Think before scrolling... The answer is `False`. By default, Python compares objects by their **memory address**, not their attribute values. `student1` and `student2` live at two different addresses in RAM, so Python sees them as two different objects — even though the data inside looks identical. (You *can* change this default behavior by defining a special `__eq__` method, which we'll touch on in the Polymorphism section.)

Quick Recap

A class is the architectural blueprint; an object is the actual house built from that blueprint in memory — each object gets its own private storage box for its attributes, and `self` is simply how a method knows which box it's currently working inside.

3.2.2 Encapsulation

The Hook (Story Mode)

September 1999. NASA loses the **Mars Climate Orbiter** — a \$125 million spacecraft — as it burns up in the Martian atmosphere. The cause? One team's software calculated thruster force in **pound-force** (imperial units). Another team's software expected the number in **newtons** (metric units). The two components exchanged this raw number directly, with no validation, no unit-checking layer, no protective interface in between.

This is exactly the kind of disaster **encapsulation** is designed to prevent. If that value had been hidden behind a protected interface — a method that validated and converted units before accepting a new value — the mismatch would have been caught instantly. Instead, a spacecraft was lost because two pieces of code touched raw, unguarded data directly.

The Explanation

Encapsulation means bundling data and the methods that operate on it into a single class, and **restricting direct outside access** to that data. Outsiders must go through controlled "doors" (methods) instead of reaching directly into the object's internals.

Everyday Analogy: Think of an ATM. You interact with a public interface — insert your card, press buttons, get cash. You never get to open the steel vault and directly rewrite the balance number stored inside the bank's server. The public buttons are your only *legal* doorway to that protected data.

Access Modifiers in Python (naming conventions, not hard rules):

Access Level	Syntax	Meaning
Public	<code>self.name</code>	Accessible from anywhere
Protected	<code>self._name</code>	Convention: "internal use only," but not enforced
Private	<code>self.__name</code>	Python performs <i>name mangling</i> to make accidental outside access hard

```
+-----+
|      BankAccount      |
+-----+
| - __balance           | <- private (data hiding)
+-----+
| + deposit(amount)     | <- public interface
| + withdraw(amount)    | <- public interface
| + get_balance()       | <- public getter
+-----+
```

Getter and Setter Methods:

A **getter** lets outside code safely *read* a private value. A **setter** lets outside code safely *change* a private value — while running validation checks first.


```

class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
        else:
            print("Deposit must be positive.")

    def withdraw(self, amount):
        if amount > self.__balance:
            print("Insufficient balance")
        else:
            self.__balance -= amount

    def get_balance(self):           # getter
        return self.__balance

    def set_balance(self, new_balance): # setter, with validation
        if new_balance >= 0:
            self.__balance = new_balance
        else:
            print("Balance cannot be negative.")

```

```

account = BankAccount(1000)
account.deposit(500)
account.withdraw(300)
print(account.get_balance())    # 1200

# Direct access is blocked by name mangling:
# print(account.__balance)      # AttributeError!

```

Why this matters: without the `deposit()` / `withdraw()` gatekeepers, someone could write `account.__balance = -99999` directly and break the entire system. Encapsulation forces every change to pass through validated logic first.

The Practical Walkthrough

Step	Code	Result
1	<code>account = BankAccount(1000)</code>	Private <code>__balance</code> set to 1000, hidden inside the object
2	<code>account.deposit(500)</code>	Passes through validation (<code>amount > 0</code>), <code>__balance</code> becomes 1500
3	<code>account.withdraw(300)</code>	Passes through validation (<code>300 <= 1500</code>), <code>__balance</code> becomes 1200
4	<code>account.__balance = -500</code>	Fails / does not touch the real data — name mangling has renamed the real attribute internally to <code>__BankAccount__balance</code> , so this creates an unrelated new attribute instead
5	<code>account.get_balance()</code>	Returns the <i>real</i> , protected value: <code>1200</code>

Interactive Stop-Point

Grab a Partner — 3.2.2: Partner A writes a `BankAccount` class with a fully **public** `balance` attribute (no underscores). Partner B plays the "hacker" and writes one line of code that sets `balance` to a negative number, instantly creating money from nothing. Now, redesign the class together using a private `__balance` attribute plus a `withdraw()` method that rejects any request larger than the current balance. Test that the exploit no longer works.

Quick Recap

Encapsulation hides an object's raw data behind protected, validated methods — like an ATM's buttons guarding the vault — so outside code can never corrupt an object's internal state directly.

3.2.3 Inheritance — Reusing and Extending Functionality

The Hook (Story Mode)

Every car ever built shares a common ancestor concept: wheels, an engine, the ability to move forward. Car manufacturers don't reinvent "what is a vehicle" from scratch every time. Instead, they start from a shared **base design** and *extend* it — adding a turbocharger for a sports car, or extra seating and a bigger battery for an electric bus.

This is precisely how **inheritance** works in code: build one solid base class, then extend it into more specific variations, without duplicating any of the shared logic.

The Explanation

Inheritance lets a new class (the **child** or **derived class**) automatically receive all the attributes and methods of an existing class (the **parent** or **base class**) — and then add or override its own extra features.

```
+-----+
|  Vehicle  |  <-- Parent (Base) Class
+-----+
| + speed   |
| + fuel    |
+-----+
| + move()  |
+-----+
      ^
      | inherits from
+-----+
| SportsCar |  <-- Child (Derived) Class
+-----+
| + turbo_boost |
+-----+
| + boost()    |
+-----+
```

Single Inheritance — one child class inherits from exactly one parent class.

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

    def move(self):
        print(f"{self.brand} is moving.")

class SportsCar(Vehicle):          # SportsCar inherits from Vehicle
    def boost(self):
        print(f"{self.brand} activates turbo boost!")

car = SportsCar("Ferrari")
car.move()    # Inherited from Vehicle: "Ferrari is moving."
car.boost()   # Defined in SportsCar: "Ferrari activates turbo boost!"
```

Using `super()` to Extend, Not Replace, the Parent:

`super()` lets a child class call the parent's version of a method — usually inside its own `__init__` — so you don't have to retype the parent's setup logic.

```

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Student(Person):
    def __init__(self, name, age, student_id):
        super().__init__(name, age) # runs Person's __init__ first
        self.student_id = student_id # then adds Student's own attribute

s = Student("Hassan", 17, "S1023")
print(s.name, s.age, s.student_id) # Hassan 17 S1023

```

Multiple Inheritance — a child class inherits from *more than one* parent class at the same time.

```

+-----+      +-----+
| Camera |      | Computer |
+-----+      +-----+
      \          /
       \        /
      +-----+
      | Smartphone |
      +-----+

```

```

class Camera:
    def take_photo(self):
        print("Photo captured.")

class Computer:
    def run_app(self):
        print("App running.")

class Smartphone(Camera, Computer):
    def call(self):
        print("Calling...")

phone = Smartphone()
phone.take_photo() # from Camera
phone.run_app()   # from Computer
phone.call()      # from Smartphone itself

```

The Practical Walkthrough

Tracing `super().__init__()` step by step for `Student("Hassan", 17, "S1023")`:

Step	What Executes	RAM State of the new object
1	<code>Student.__init__</code> starts running with <code>self</code> bound to new object	Object exists, but empty
2	<code>super().__init__(name, age)</code> is called	Control jumps <i>up</i> to <code>Person.__init__</code>
3	Inside <code>Person.__init__</code> : <code>self.name = "Hassan"</code>	<code>name: "Hassan"</code> added to the object
4	Inside <code>Person.__init__</code> : <code>self.age = 17</code>	<code>age: 17</code> added to the object
5	Control returns to <code>Student.__init__</code>	Continues from where it left off
6	<code>self.student_id = "S1023"</code>	<code>student_id: "S1023"</code> added to the object
Final Object		<code>{ name: "Hassan", age: 17, student_id: "S1023" }</code>

Multilevel Inheritance (a chain, like a family tree — Grandparent → Parent → Child):

```
Vehicle ---> Car ---> ElectricCar
```

```
class Vehicle:
    def start_engine(self):
        print("Engine started.")

class Car(Vehicle):
    def drive(self):
        print("Car is driving.")

class ElectricCar(Car):
    def charge(self):
        print("Battery is charging.")

tesla = ElectricCar()
tesla.start_engine() # inherited from Vehicle (grandparent)
tesla.drive()       # inherited from Car (parent)
tesla.charge()       # defined in ElectricCar itself
```

Interactive Stop-Point

Grab a Partner — 3.2.3: Build a `Vehicle` base class with `speed` and a `move()` method. Partner A extends it into a `Car` subclass with a unique `honk()` method. Partner B extends it into a `Bike` subclass with a unique `ring_bell()` method. Now both of you instantiate one object of each and call every method — inherited and new — to prove both subclasses share the base functionality but each keeps its own unique behavior.

Quick Recap

Inheritance lets a child class automatically absorb all the attributes and methods of a parent class — through single, multiple, or multilevel chains — so you extend functionality instead of rewriting it from scratch, with `super()` as your bridge back to the parent's setup logic.

3.2.4 Polymorphism

The Hook (Story Mode)

Look at any modern remote control, or the play button on your phone. One single button —  — plays a song in Spotify, plays a video on YouTube, or resumes an animation inside a game. The button's *label and position never change*, but what actually happens when you press it depends entirely on which app is currently running.

That is polymorphism: one shared interface, many different real behaviors underneath, depending on context.

The Explanation

Polymorphism ("many forms") means the *same* method name can behave *differently* depending on which object it's called on.

Method Overriding — Runtime Polymorphism:

A child class provides its own version of a method that already exists in its parent class. Python decides *which* version to run while the program is actually executing (at runtime) — based on the real class of the object.

```

class Shape:
    def draw(self):
        return "Drawing a generic shape"

class Circle(Shape):
    def draw(self):
        return "Drawing a circle ◦"

class Square(Shape):
    def draw(self):
        return "Drawing a square ◻"

class Triangle(Shape):
    def draw(self):
        return "Drawing a triangle ◀"

shapes = [Circle(), Square(), Triangle(), Shape()]

for shape in shapes:
    print(shape.draw())

```

Output:

```

Drawing a circle ◦
Drawing a square ◻
Drawing a triangle ◀
Drawing a generic shape

```

The loop calls `.draw()` on every item using *identical* code — but each object answers with its own overridden behavior. This is exactly why polymorphism scales so well: adding a `Pentagon` class tomorrow requires **zero changes** to the loop above. Just define `Pentagon(Shape)` with its own `draw()`, and it slots right in.

Duck Typing — "If it walks like a duck and quacks like a duck..."

Python doesn't require objects to formally share a parent class to be treated the same way — it only cares whether the object *has* the method you're trying to call. This flexible style is called **duck typing**.

```

class Duck:
    def sound(self):
        return "Quack!"

class Dog:
    def sound(self):
        return "Bark!"

def make_it_speak(animal):
    print(animal.sound())    # Python doesn't care about the class,
                            # only that .sound() exists

make_it_speak(Duck())      # Quack!
make_it_speak(Dog())      # Bark!

```

Compile-time Polymorphism (Method "Overloading") in Python:

Languages like Java or C++ let you define multiple versions of a method with different parameter counts, chosen at compile time. Python is dynamically typed and doesn't need this — instead, you mimic the same flexibility using default arguments or `*args`:

```

class Calculator:
    def multiply(self, a=1, b=1, *args):
        result = a * b
        for num in args:
            result *= num
        return result

calc = Calculator()
print(calc.multiply(4))      # 4
print(calc.multiply(2, 3, 4)) # 24

```

Special / Magic Methods — Making Your Own Objects Behave Like Built-in Types:

Python's built-in operators (`+`, `len()`, `print()`) are themselves powered by polymorphism — through special methods surrounded by double underscores ("dunder" methods).


```

class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):
        return f'"{self.title}" ({self.pages} pages)'

    def __len__(self):
        return self.pages

    def __add__(self, other):
        return self.pages + other.pages

b1 = Book("Python Basics", 220)
b2 = Book("Advanced OOP", 340)

print(b1)          # calls __str__ -> 'Python Basics' (220 pages)
print(len(b1))     # calls __len__ -> 220
print(b1 + b2)     # calls __add__ -> 560

```

This is also how `student1 == student2` from section 3.2.1 could be made to compare *attribute values* instead of memory addresses — by defining a custom `__eq__` method.

The Practical Walkthrough

Tracing the `shapes` loop, line by line:

Iteration	shape object	shape.draw() looks first at...	Method actually run	Output
1	<code>Circle()</code>	<code>Circle</code> class — found <code>draw()</code> there	<code>Circle.draw</code>	"Drawing a circle o"
2	<code>Square()</code>	<code>Square</code> class — found <code>draw()</code> there	<code>Square.draw</code>	"Drawing a square □"
3	<code>Triangle()</code>	<code>Triangle</code> class — found <code>draw()</code> there	<code>Triangle.draw</code>	"Drawing a triangle △"
4	<code>Shape()</code>	<code>Shape</code> class itself (no override to look for)	<code>Shape.draw</code>	"Drawing a generic shape"

What just happened conceptually? Python always checks the *actual* class of the object first for an overridden method, and only falls back to the parent's version if the child never redefined it. This lookup happens fresh, every single time, at runtime.

Interactive Stop-Point

Pause & Think — 3.2.4: Why is method overriding (one `draw()` method per shape class) better design than writing three separate standalone functions — `draw_circle()`, `draw_square()`, `draw_triangle()` — and calling the right one with a big `if/elif` chain? Specifically: what happens to your `if/elif` chain when a 4th shape, `Pentagon`, is added? What happens to the polymorphic loop above?

Hint: With overriding, adding `Pentagon(Shape)` requires touching **zero** existing code outside the new class itself. With the `if/elif` approach, you must find and edit that chain every single time a new shape is added — a common source of forgotten bugs in large systems.

Quick Recap

Polymorphism lets the *same* method call — like `.draw()` or `.sound()` — automatically trigger different, object-specific behavior at runtime, making your code flexible enough to support brand-new object types without rewriting existing logic.

Chapter Glossary (Quick Reference)

Term	Plain-English Definition
Class	A blueprint that defines attributes and methods for a type of object
Object	A specific instance created from a class, with its own real data
Attribute	A variable that belongs to an object
Method	A function that belongs to a class
Instantiation	The act of creating an object from a class
Constructor (<code>__init__</code>)	The special method that runs automatically when an object is created
<code>self</code>	A reference to "this specific object" inside a method
Encapsulation	Bundling data and methods together, and hiding/protecting data from direct outside access
Getter / Setter	Methods used to safely read or safely change a private attribute
Inheritance	A mechanism where a child class reuses and extends a parent class's attributes/methods
<code>super()</code>	A function used to call a parent class's method (often <code>__init__</code>) from inside a child class
Polymorphism	The ability of the same method name to behave differently depending on the object calling it
Method Overriding	A child class providing its own version of a method already defined in the parent
Duck Typing	Python's style of caring only whether an object <i>has</i> a needed method, not its exact class
Magic / Special Methods	Double-underscore methods (<code>__str__</code> , <code>__len__</code> , <code>__add__</code>) that let custom objects work with built-in operators and functions

Chapter Summary

Object-Oriented Programming reorganizes code around real-world "things" instead of a straight line of instructions. **Classes** are blueprints; **objects** are the real, independent instances built from them, each with its own private memory. **Encapsulation** protects an object's internal data behind

validated public methods, preventing the kind of raw, unguarded data exchange that once destroyed a Mars spacecraft. **Inheritance** lets you build a solid base class once and extend it — through single, multiple, or multilevel chains — into more specialized subclasses without duplicating code. **Polymorphism** lets one shared method name, like `.draw()` or the ▶ button, trigger different real behavior depending on the object underneath, keeping your code flexible and effortlessly extendable.

Together, these four pillars — classes/objects, encapsulation, inheritance, and polymorphism — are the same toolkit used to build everything from Fortnite's character system to Instagram's user accounts to the payment app on your phone. You now speak that language.